

Laxmi Charitable Trust's
Sheth L.U.J College of Arts & Sir M.V. College of Science and Commerce
Department of Information Technology (B.Sc.I.T Semester III)
Scala For Data Science

Practical XIX

Roll No.: S021	Name: LODHI IQRA
Class: SYBsc IT	Subject: Scala For Data Science
Date of Assignment: 18/08/2026	Date/Time of Submission: 18/08/2026

AIM: Combine two plots (e.g., scatter and line plot) in a single visualization using Breeze-viz.

CODE:

```
1  import breeze.linalg._
2  import breeze.plot._
3
4  object IQRA extends App{
5      val x = DenseVector(1.0,2.0,3.0,4.0,5.0)
6      val yscat = DenseVector(2.0,4.0,1.0,8.0,7.0)
7      val yline = DenseVector(1.5,3.0,4.5,6.0,7.5)
8
9      val fig = Figure()
10     val plt = fig.subplot(0)
11
12     plt += plot(x,yscat, '.', colorcode="blue")
13     plt += plot(x,yline, '-', colorcode="green")
14     plt.title = "Scatter + Line Plot"
15     plt.xlabel = "X-axis"
16     plt.ylabel = "Y- axis"
17     fig.refresh()
18     println("IQRA S021")
19
20 }
```

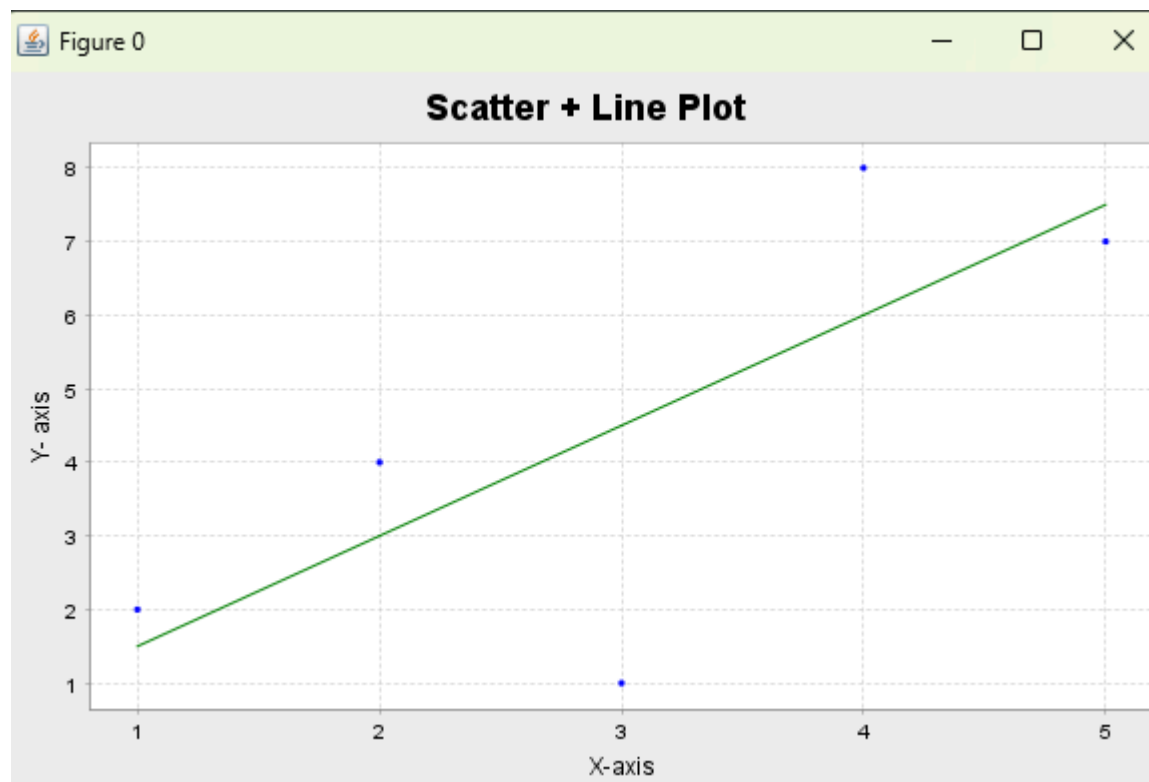
IQRA

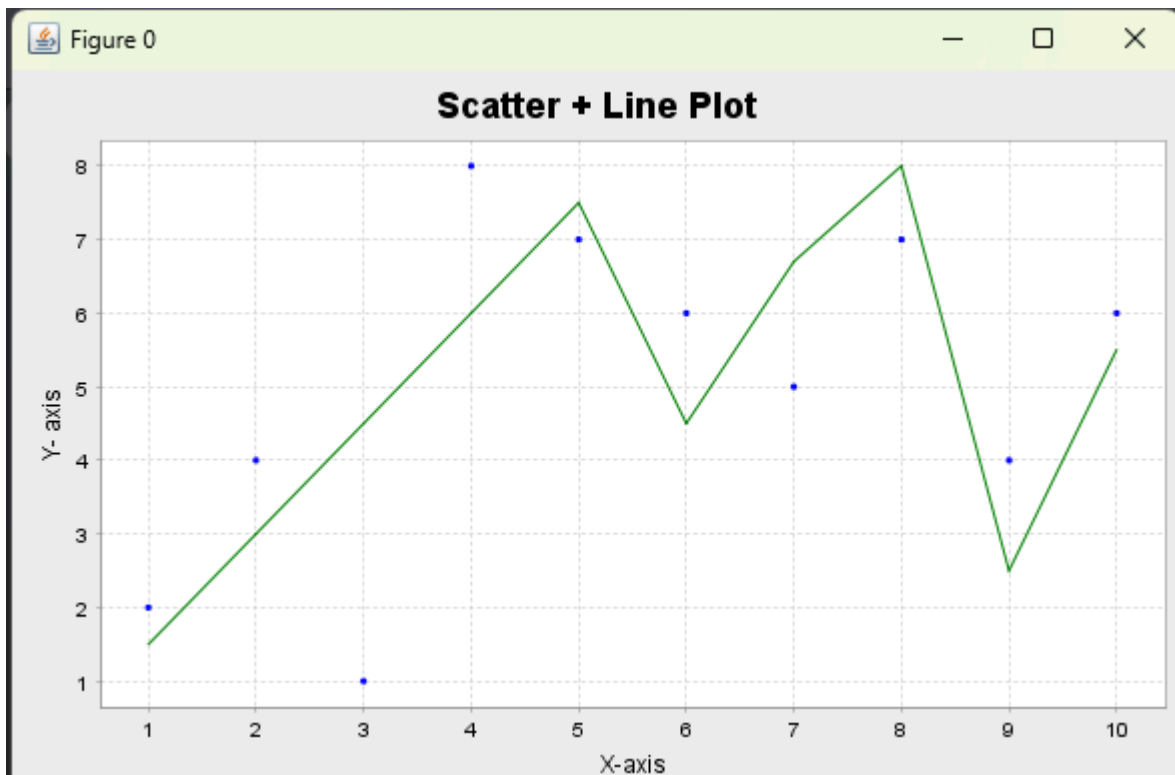
```

1  import breeze.linalg._
2  import breeze.plot._
3
4  object IQRA extends App{
5      val x = DenseVector(1.0,2.0,3.0,4.0,5.0,6.0,7.0,8.0,9.0,10.0)
6      val yscat = DenseVector(2.0,4.0,1.0,8.0,7.0,6.0,5.0,7.0,4.0,6.0)
7      val yline = DenseVector(1.5,3.0,4.5,6.0,7.5,4.5,6.7,8.0,2.5,5.5)
8
9      val fig = Figure()
10     val plt = fig.subplot(0)
11
12     plt += plot(x,yscat, '.', colorcode ="blue")
13     plt += plot(x,yline, '-', colorcode ="green")
14     plt.title = "Scatter + Line Plot"
15     plt.xlabel = "X-axis"
16     plt.ylabel = "Y- axis"
17     fig.refresh()
18     println("IQRA S021")
19
20 }

```

OUTPUT:





Practical XX

AIM: Read a CSV file and calculate basic statistics for each numeric column. Use the scala-csv library or similar tools.

CODE:

```

1 import scala.io.Source
2
3 object IQRA extends App{
4     val file = Source.fromResource("iris.csv")
5     val data = file.getLines()
6         .drop(1)
7         .map(line => line.split(",")(0).toDouble)
8         .toList
9
10    file.close()
11
12    val count = data.length
13    val mean = data.sum/count
14    val min = data.min
15    val max = data.max
16
17    val variance = data.map(x => math.pow(x - mean, 2)).sum / count
18    val std = math.sqrt(variance)
19
20    println(s"Count: $count")
}

```

IQRA

OUTPUT:

```

sepal_length,sepal_width,petal_length,species
5.1,3.5,1.4,0.2,setosa
4.9,3.0,1.4,0.2,setosa
4.7,3.2,1.3,0.2,setosa
4.6,3.1,1.5,0.2,setosa
5.0,3.6,1.4,0.2,setosa
5.4,3.9,1.7,0.4,setosa
4.6,3.4,1.4,0.3,setosa
5.0,3.4,1.5,0.2,setosa
4.4,2.9,1.4,0.2,setosa
4.9,3.1,1.5,0.1,setosa
5.4,3.7,1.5,0.2,setosa
4.8,3.4,1.6,0.2,setosa
4.8,3.0,1.4,0.1,setosa
4.3,3.0,1.1,0.1,setosa
5.8,4.0,1.2,0.2,setosa
5.7,4.4,1.5,0.4,setosa
5.4,3.9,1.3,0.4,setosa
5.1,3.5,1.4,0.3,setosa
5.7,3.8,1.7,0.3,setosa

```

```
"C:\Program Files\Java\jdk1.8.0_181\bin\java
Count: 150
Mean: 5.84
Minimum: 4.30
Maximum: 7.90
Standard Deviation: 0.83
IQRA S021

Process finished with exit code 0
```

Practical XXI

AIM: Join two CSV files in Spark DataFrames based on a common column and write the output to a file

CODE:

```
ThisBuild / version := "0.1.0-SNAPSHOT"

ThisBuild / scalaVersion := "2.12.18"

libraryDependencies ++= Seq(
  "org.apache.spark" %% "spark-core" % "3.5.1",
  "org.apache.spark" %% "spark-sql" % "3.5.1"
)

lazy val root = (project in file("."))
  .settings(
    name := "IQRA"
  )
```

```

3  object IQRA extends App {
4      val s = SparkSession.builder()
5          .appName("JoinTwoCSV")
6          .master("local[*]")
7          .getOrCreate()
8
9      val d1 = s.read
10         .option("header", "true")
11         .csv("file1.csv")
12
13     val d2 = s.read
14         .option("header", "true")
15         .csv("file2.csv")
16
17     val j = d1.join(d2, Seq("id"), "inner")
18
19     j.show()
20
21     j.write.option("header", "true")
22
23     s.stop()
24     println("IQRA S021")
25 }

```

Create 2 csv file:

id,name	id,score
1,DIVYESH	1,85
2,DHARMI	2,45
3,SMIT	3,88
4,URVISH	4,78
5,HEMISH	5,67

OUTPUT:

```

26/08/18 11:41:42 INFO SparkContext: SparkContext is available with 20 executors
+---+-----+-----+
| id|   name|score|
+---+-----+-----+
|  1|DIVYESH|  85|
|  2| DHARMI|  45|
|  3|   SMIT|  88|
|  4|  URVISH|  78|
|  5|  HEMISH|  67|
+---+-----+-----+

26/08/18 11:41:42 INFO SparkUI: Stopped Spark at 26/08/18 11:41:42
26/08/18 11:41:42 INFO MapOutputTrackerMaster: MapOutputTrackerMaster is available
26/08/18 11:41:42 INFO MemoryStore: MemoryStore is available

```